# Guia de desenvolvimento e validação da N2

PMI III • ADS31 • Orientações às equipes • 02/10/2026

A N2 valida a evolução do projeto após a N1. Cada equipe desenvolverá funcionalidades executáveis nas Sprints 1 e 2 e manterá requisitos, modelos, arquitetura, código e testes coerentes. Utilize este guia com os três templates de entrega.

## Calendário de trabalho

| Marco | Prazo | Resultado esperado |
|---|---|---|
| Sprint 1 | 16/10/2026 | Primeiro incremento funcional e dossiê atualizado. |
| Sprint 2 | 30/10/2026 | Evolução integrada e testes das funcionalidades anteriores. |
| N2 final | 06/11/2026 | Entrega e apresentação presencial do sistema e das evidências. |

## Como usar os materiais

1. Leia os passos deste guia. 2. Abra o template da Sprint correspondente. 3. Preencha a ficha inicial antes de selecionar cada história. 4. Atualize a mesma ficha durante o desenvolvimento. 5. Registre testes e resultado ao fechar a Sprint. 6. Consolide as duas entregas no template final.

Os campos [Preencher] são editáveis. Duplique as tabelas de registros e a ficha por história conforme a necessidade. Use “Não se aplica” com justificativa nos campos que não correspondem à solução. Os exemplos são ilustrativos e devem ser substituídos pelos dados reais do projeto.

## O que será observado

Funcionamento, coerência técnica, qualidade dos testes, evidências do processo e domínio da equipe. Este material não atribui pesos nem estabelece quantidade fixa de histórias. O escopo deve caber na capacidade da equipe e produzir valor demonstrável.

# Passos 1 e 2 Retomada e planejamento

## Passo 1 Tratar o feedback da N1

Liste cada ajuste recebido, o artefato afetado, a ação, o responsável e a evidência de resolução. Atualize os requisitos e critérios de aceite antes de estimar o trabalho. Mantenha os identificadores já usados na N1.

## Passo 2 Definir objetivo e escopo da Sprint

Escreva um objetivo que descreva um resultado para o usuário. Selecione histórias prioritárias, considere disponibilidade, dependências e estimativas. Desdobre as histórias em tarefas de modelagem, código, integração e teste. Revise o plano da Sprint 2 após os resultados da Sprint 1.

| Exemplo didático | Registro |
|---|---|
| Objetivo da Sprint 1 | Permitir que o solicitante registre e consulte uma solicitação. |
| História US03 | Como solicitante, quero cadastrar uma solicitação para acompanhar meu pedido. |
| Critério CA01 | Com título e descrição válidos, o sistema registra a solicitação e a exibe na lista. |
| Critério CA02 | Sem título, o sistema informa a pendência e não grava a solicitação. |
| RNF ilustrativo | Na base de teste com 100 registros, a listagem responde em até 2 segundos no ambiente documentado. |

## Ficha inicial para entrada na Sprint

[ ] Necessidade e requisito relacionado identificados.

[ ] Critérios de aceite verificáveis e cenários de falha definidos.

[ ] Modelagem mínima pertinente e abordagem técnica inicial registradas.

[ ] Dependências, estimativa e capacidade discutidas pela equipe.

Esta regra de entrada é uma convenção didática da disciplina. O dossiê completo se desenvolve durante a Sprint. Não é necessário antecipar todos os detalhes do código.

# Passos 3 e 4 Modelagem e arquitetura

# Passo 3 Atualizar os modelos pertinentes

Refine atores e objetivos no Diagrama de Casos de Uso. Identifique as histórias abrangidas. No Diagrama de Sequência, mostre participantes reais, ordem das mensagens, retorno e alternativas. O CRUD central exige representação do fluxo implementado. Inclua Login quando estiver no escopo e identifique autenticação simulada.

| Abordagem | O que documentar |
|---|---|
| SQL | DER com chaves e cardinalidades, tabelas, restrições e scripts ou migrações. Relacione entidades do código às tabelas. Inclua ORM se utilizado. |
| NoSQL | Exemplo de documento, tipos, campos obrigatórios, referências ou dados embutidos, validações e índices pertinentes. Inclua schema Mongoose se utilizado. |
| Sem banco | Estrutura JSON, identificadores, estado da aplicação, chaves de armazenamento e regras de leitura, gravação e recuperação. Diferencie estado em memória de persistência local. |

# Passo 4 Explicar a arquitetura

A equipe deve apresentar como o sistema está organizado, quais componentes fazem parte da solução e como eles se comunicam para atender às funcionalidades desenvolvidas.

O objetivo é demonstrar que existe coerência entre os requisitos, a modelagem, o código e a forma como os dados são processados e armazenados.

**4.1 Identifique os componentes utilizados**

Apresente somente os componentes que realmente fazem parte do projeto:

- **Front-end:** telas e interação com o usuário;
- **Back-end:** regras de negócio e processamento;
- **Persistência:** banco SQL, banco NoSQL, LocalStorage ou arquivo;
- **Serviços externos:** APIs, autenticação, mapas, pagamentos ou outros serviços utilizados.

A equipe não deve incluir componentes que não foram implementados.

**Exemplos de arquiteturas**

**Solução com front-end, back-end e banco de dados**

Usuário

↓

Front-end

↓

API / Back-end

↓

Banco de dados

Explique:

- Onde o usuário interage com o sistema;
- Onde as regras de negócio são executadas;
- Como os dados são enviados;
- Onde os dados são armazenados;
- Como o resultado retorna ao usuário.

**Solução sem back-end**

Usuário

↓

Interface da aplicação

↓

Funções de validação

↓

LocalStorage ou estado local

Nesse caso, explique que:

- O sistema funciona localmente;
- Não existe servidor próprio;
- As regras são executadas no navegador;
- Os dados podem ser armazenados localmente;
- A solução possui limitações para compartilhamento entre usuários.

**Solução com banco NoSQL**

Front-end

↓

Serviço ou API

↓

Coleções de documentos

Explique:

- Como os documentos são estruturados;
- Quais campos são obrigatórios;
- Como ocorre a consulta e atualização;
- Como as validações são realizadas.

**4.2 Registre uma decisão técnica**

Escolha uma decisão relevante da arquitetura e registre-a no Dossiê Técnico da Sprint.

Utilize a seguinte estrutura:

| **Item** | **O que deve ser informado** |
|---|---|
| Contexto | Qual problema ou necessidade precisava ser resolvido? |
| Alternativas | Quais opções foram consideradas? |
| Escolha | Qual alternativa foi adotada? |
| Justificativa | Por que essa opção foi escolhida? |
| Consequência | Quais benefícios, limitações ou impactos foram gerados? |

**Exemplo**

**Contexto:** o sistema precisa validar regras de negócio e compartilhar os dados entre diferentes usuários.

**Alternativas consideradas:**

- Validar as informações somente no front-end;
- Criar uma API no back-end;
- Utilizar apenas o LocalStorage.

**Escolha:** utilizar uma API no back-end.

**Justificativa:** o back-end centraliza as regras de negócio e permite armazenar os dados em um banco compartilhado.

**Consequência:** a solução exige servidor e configuração adicional, mas permite maior controle, segurança e integração entre os usuários.

**4.3 Descreva as interfaces de comunicação**

Para cada interface importante, registre como ocorre a comunicação entre os componentes.

Uma interface pode ser:

- Uma API;
- Uma função;
- Um formulário;

- Uma operação de consulta;
- Uma operação de gravação;
- Uma chamada entre módulos.

Para cada interface, informe:

1. Nome da interface;
2. Dados de entrada;
3. Validações realizadas;
4. Resultado de saída;
5. Erros possíveis.

**Exemplo de API**

POST /solicitacoes

| **Elemento** | **Descrição** |
|---|---|
| Entrada | Título e descrição da solicitação |
| Validações | Título obrigatório e descrição válida |
| Saída | Identificador da solicitação criada |
| Sucesso | Registro armazenado no banco |
| Erro | Mensagem informando dados inválidos |

**Exemplo de função local**

cadastrarSolicitacao(dados)

| **Elemento** | **Descrição** |
|---|---|
| Entrada | Objeto contendo título e descrição |
| Validações | Verificação do preenchimento do título |
| Saída | Objeto com identificador e status |
| Sucesso | Dados gravados no LocalStorage |
| Erro | Mensagem de validação para o usuário |

**4.4 Relacione a arquitetura com uma história**

Escolha uma história implementada e demonstre o caminho completo da informação:

Usuário

→ Interface

→ Regra de negócio

→ Persistência

→ Retorno do resultado

A equipe deve explicar:

- Qual história está sendo analisada;
- Qual componente recebe os dados;
- Onde ocorre a validação;
- Onde o dado é armazenado;
- Qual resposta é apresentada ao usuário;
- Como o teste comprova que o fluxo funciona.

**Entrega esperada do Passo 4**

**A equipe deverá entregar:**

- **Diagrama da arquitetura real do sistema;**
- **Descrição da responsabilidade de cada componente;**
- **Fluxo de comunicação de uma história implementada;**
- **Registro de uma decisão técnica;**
- **Descrição de pelo menos uma interface;**
- **Validações e erros previstos;**
- **Relação entre arquitetura, requisitos, código e testes.**

**Pergunta de verificação**

Outra equipe conseguiria olhar para a arquitetura e entender onde o dado entra, onde é validado, onde é armazenado e qual resultado retorna?

**Se a resposta for negativa, o modelo precisa ser revisado antes da inclusão da história na Sprint.**

# Passos 5 e 6 Implementação e testes

## Passo 5 Desenvolver com integração contínua do trabalho

Use feature/US03-cadastro para a história, develop para integrar alterações e main para a versão estável demonstrável. Registre commits que expliquem mudanças. Revise a integração por pull request ou registro equivalente. Esta organização é o padrão didático desta etapa, não uma exigência do Scrum.

Atualize o quadro com trabalho a fazer, em andamento, em revisão e concluído. Registre bloqueios com ação e responsável. Relacione história, tarefas, arquivos e commits. A contribuição de cada integrante deve ser explicável, sem usar quantidade de commits como medida isolada de desempenho.

## Passo 6 Testar os critérios e as integrações

Execute cenários válidos e inválidos. Para cada teste, informe pré-condição, passos, dados, resultado esperado, resultado observado, situação e evidência. Capturas ajudam, mas devem mostrar contexto e resultado. Inclua testes automatizados quando viáveis e explique como executá-los.

| Teste didático | Esperado | Evidência |
|---|---|---|
| T01 • cadastrar com dados válidos | Solicitação salva e visível na lista. | Resultado real, captura ou log e versão testada. |
| T02 • enviar sem título | Mensagem de validação e nenhuma gravação. | Resultado real e confirmação de ausência do registro. |
| T03 • recarregar a aplicação | Dados persistidos reaparecem quando a solução prevê persistência. | Passos e observação após recarregamento. |

## Definição de concluído da etapa

[ ] Critérios de aceite verificados e falhas impeditivas resolvidas.

[ ] Código integrado e executável na versão registrada.

[ ] Modelos e dossiê correspondem à implementação.

[ ] Evidências acessíveis e instruções de execução testadas.

# Passos 7 e 8 Fechamento e validação

## Passo 7 Fechar cada Sprint

Compare planejado e realizado. Mostre o incremento, registre feedback e atualize o backlog. Identifique histórias incompletas e o trabalho restante. Não apresente uma história incompleta como concluída. Na retrospectiva, registre um problema do processo e uma ação concreta para a próxima etapa.

Em 16/10, envie o template da Sprint 1 e os links da versão correspondente. Em 30/10, envie o template da Sprint 2, incluindo o tratamento do feedback e testes das funcionalidades já existentes. Use um snapshot dos documentos ou links vinculados à versão para preservar cada marco.

## Passo 8 Consolidar e apresentar a N2

Entre 30/10 e 06/11, estabilize o incremento, corrija falhas, revise testes e organize o dossiê final. Em 06/11, entregue o template final e apresente o sistema rodando em sala. Demonstre uma história completa e explique sua ligação com requisito, modelos, código e testes.

[ ] Repositório acessível, versão final identificada e links testados.

[ ] README com dependências, configuração, comandos e dados de demonstração.

[ ] Configurações de exemplo sem senhas ou credenciais reais.

[ ] Roteiro com cenário de sucesso e cenário de falha.

[ ] Pendências, limitações e contribuições individuais registradas.

## Organização das entregas

Sugestão de nomes: Time_Sprint1_16_10_2026; Time_Sprint2_2026-10-30; Time_N2_06_11_2026. Mantenha código no repositório e anexos de modelagem em pasta identificada. No documento, informe os links e a versão usada. Entregue nos espaços correspondentes disponibilizados no Classroom.

## Referência conceitual

Schwaber, K.; Sutherland, J. Scrum Guide, 2020. https://scrumguides.org/scrum-guide.html. O dossiê, as fichas e o fluxo de branches são convenções didáticas desta disciplina. Pesos e tempo de apresentação serão informados pelo professor.